

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Curso:

Base de Datos II

Laboratorio 05: Hash File

Alumnos:

Llerena Silva, Nicolás Alejandro

(202110190)

Gonzalez Kremer ,Mauricio Gabriel

(202120239)

Profesor:

Lovon, Percy

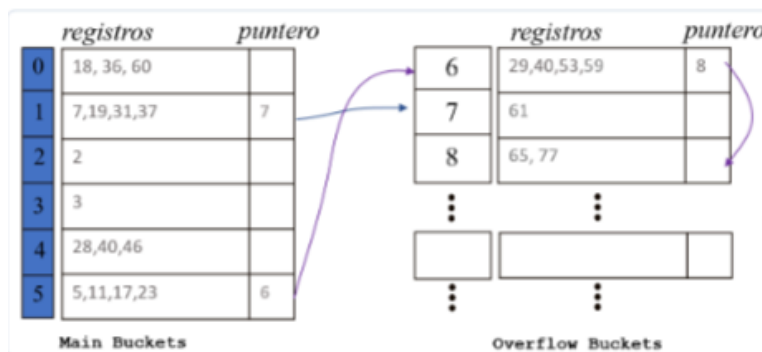
Lima, setiembre de 2026

Laboratorio 05

P1. Static Hashing

Implementar Static Hashing con buckets y desbordamiento encadenado.

1. Diseñar la estructura del bucket y la clase que gestiona el archivo.
2. Inserción: Si el bucket está lleno, crear y enlazar un bucket overflow.
3. Búsqueda: Localizar el bucket y buscar el registro, incluyendo buckets overflow.
4. Eliminación: Eliminar el registro. ¿Qué hacer si un bucket queda vacío?



Código en python:

```
class Record:
    def __init__(self, key, value):
        self.key = key
        self.value = value

    def __repr__(self):
        return f"({self.key}: {self.value})"

class Bucket:
    def __init__(self, capacity=2):
        self.capacity = capacity
```

```

        self.records = []
        # Puntero al siguiente bucket de desbordamiento
        self.next_overflow = None

    def is_full(self):
        return len(self.records) >= self.capacity

class StaticHashFile:
    def __init__(self, num_buckets=3, bucket_capacity=2):
        self.num_buckets = num_buckets
        self.bucket_capacity = bucket_capacity
        # Creacion de los buckets principales
        self.buckets = [Bucket(bucket_capacity) for _ in
range(num_buckets)]

    def _hash(self, key):
        # Funcion hash simple mediante operador módulo
        return key % self.num_buckets

    def insert(self, key, value):
        h = self._hash(key)
        current = self.buckets[h]

        # Recorrer la cadena, bucket principal y desbordamientos
        while True:
            # Si key ya existe, actualizar su valor
            for r in current.records:
                if r.key == key:
                    r.value = value
                    return

            # Si hay espacio en el bucket actual, insertar elemento
            if not current.is_full():
                current.records.append(Record(key, value))
                return

            # Si el bucket esta lleno y no hay overflow, crear uno nuevo y
enlazamos
            if current.next_overflow is None:

```

```

        current.next_overflow = Bucket(self.bucket_capacity)
        current.next_overflow.records.append(Record(key, value))
        return

    # Avanza al siguiente bucket de desbordamiento
    current = current.next_overflow

def search(self, key):
    h = self._hash(key)
    current = self.buckets[h]

    # Buscar recorriendo la cadena de buckets
    while current is not None:
        for r in current.records:
            if r.key == key:
                return r.value
        current = current.next_overflow
    # No se encontro
    return None

def delete(self, key):
    h = self._hash(key)
    current = self.buckets[h]

    # Buscar el registro en la cadena para eliminarlo
    while current is not None:
        for i, r in enumerate(current.records):
            if r.key == key:
                del current.records[i]
                return True
        current = current.next_overflow

    return False

```

P2. Extendible Hashing

Aplicar el algoritmo de Extendible Hashing al siguiente conjunto de claves:

keys: 10, 13, 34, 6, 23, 12, 15, 73, 28, 19, 67, 17, 41, 87, 57, 27, 11

a) Proceso de inserción de datos

- Para calcular la posición del bucket, utilice la función:

$$\text{Pos_bucket} = \text{Binary}(\text{hash}(\text{key}) \% 2^D)$$

- Factor de bloque: $\text{FB} = 3$
- La profundidad global inicia en $D = 2$ y se incrementa cada vez que se realiza un rehashing ($D = 3, D = 4 \dots$)
- El índice hash debe direccionar a los buckets correspondientes en el archivo de datos. El archivo inicia con dos buckets (0 y 1) y se incrementa el número de buckets cuando ocurre un overflow.
- Cuando un bucket se desborda, aplique Split y actualice el índice con la nueva entrada referenciando al bucket creado. Si se alcanza la profundidad global, encadene los buckets desbordados.
- Mantenga un encadenamiento máximo de $K = 1$; es decir, solo se permite un bucket encadenado. Si se supera, aplique rehashing.

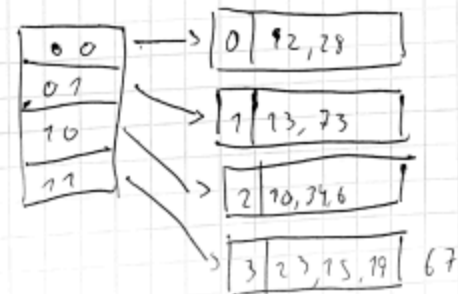
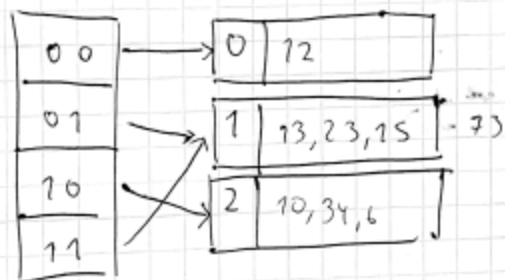
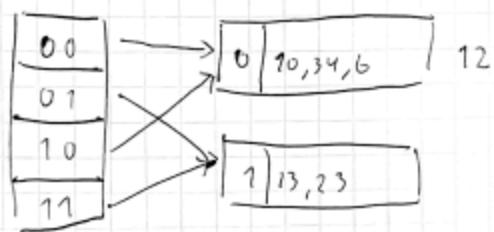
Resolver gráficamente paso a paso.

Extendible Hashing

$FH = 3$ $D = 2$

$$2^D = 4$$

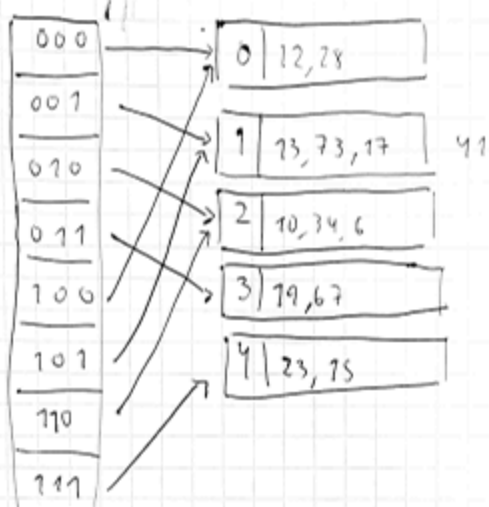
$FH = 3$



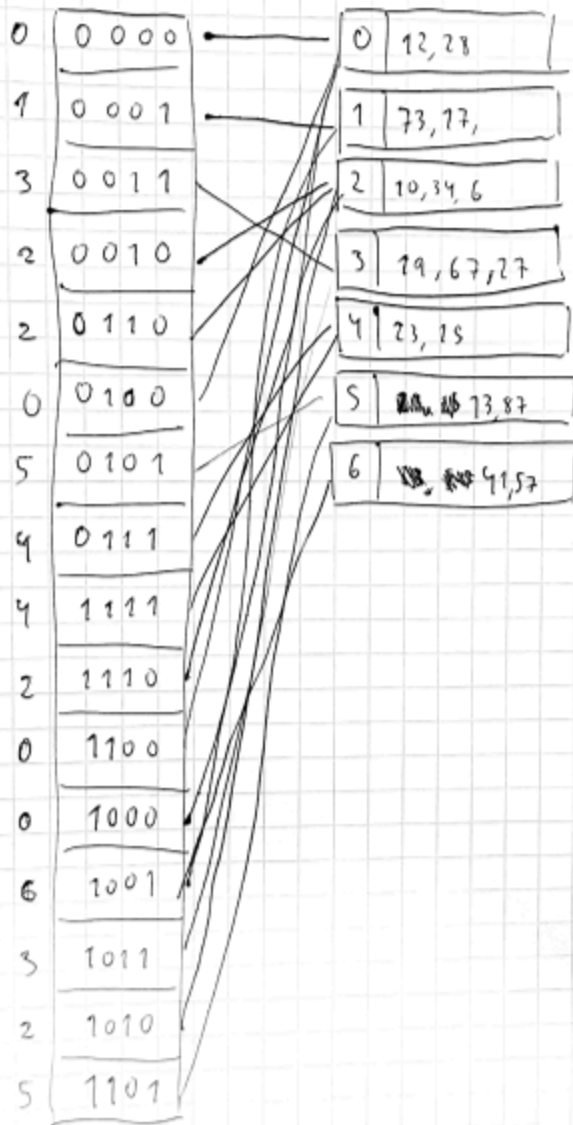
1010	10
1101	13
10010	34
110	6
10111	23
1100	12
1111	15
1010001	73
11100	28
10011	19
1000011	67
10001	17
101001	41
1010111	87
111001	57
11011	27
1011	11

$F0 = 3$ $0 = 3$

$2^0 = 8$



F 0 = 3 0 = 4



$PB = 3$		$D = 4$	
0	0000	0	12, 28
7	0001	1	73, 17
3	0011	2	10, 34, 6
2	0010	3	19, 67
2	0110	4	23, 15
0	0100	5	13, 82
5	0101	6	41, 57
4	0111	7	27, 11
4	1111		
2	1110		
0	1100		
0	1000		
6	1001		
7	1011		
2	1010		
5	1101		

b) Algoritmo de búsqueda

- Proponga el algoritmo de alto nivel necesario para la búsqueda de un registro, considerando todos los casos posibles.

Usamos la función hash al elemento a insertar para obtener el número del bucket al que debe ser insertado.

Seguirá buscando el bucket al que será insertado el elemento usando el índice obtenido.

Inicializar un puntero temporal en el bucket principal y repetir el siguiente proceso mientras el puntero sea nulo (None)

Procedemos a buscar dentro de la lista de registros del bucket actual si alguna clave coincide con la clave buscada.

Caso 1: Si el elemento a buscar es encontrado, entonces devolver el elemento y terminar el algoritmo.

Caso 2: Si el elemento no esta encontrado en el bucket actual, entonces se se buscará en el bucket de desbordamiento (siguiente_overflow) y continuar la búsqueda.

Caso 3: Si el puntero llega a None (Fin de la cadena) sin encontrar la clave, entonces concluir que el registro no existe en la estructura y retornar un “No encontrado”

c) Algoritmo de eliminación

- Proponga el algoritmo de alto nivel para eliminar un registro, considerando todos los casos.
¿Cómo gestionar los buckets libres?

Primero se deberá de buscar el registro, aplicamos los pasos iniciales del algoritmo de búsqueda.

Si el registro no se encuentra, hasta el bucket el bucket de desencadenamiento, entonces no se elimina nada.

En el caso en el que sí encuentre el registro, este lo elimina dicha clave.

En el caso que queden buckets libres, primero se debe evaluar qué bucket es el que se busca eliminar. Si es un bucket principal entonces solo quedará vacío. Pero si es un bucket de desencadenamiento entonces, el puntero del bucket previo deberá cambiar para apuntar al bucket siguiente.

P3. Trie Hashing

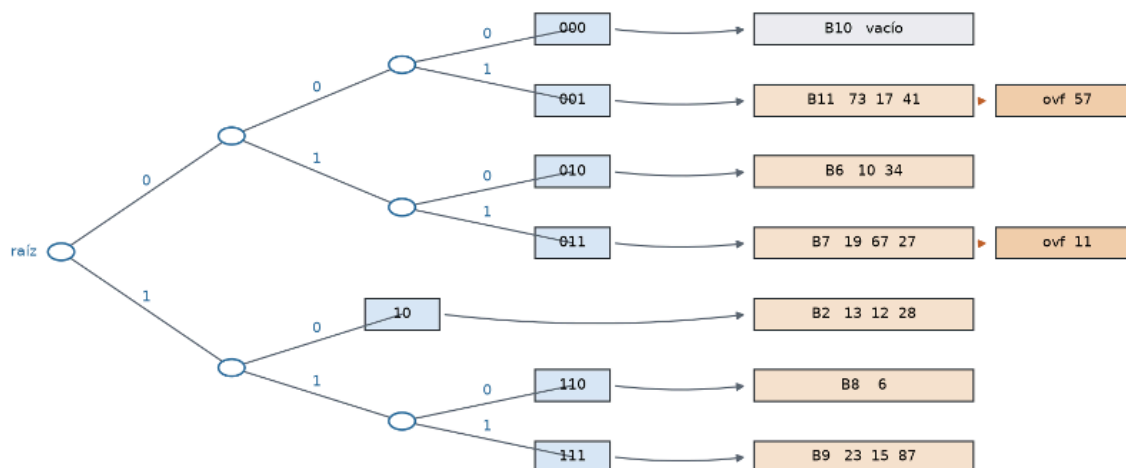
Resolver manualmente el caso anterior, pero usando un Trie (árbol digital) para mantener el directorio como un árbol binario, donde los nodos hoja guardan los punteros a los buckets en el archivo de datos.

Trie Hashing, Litwin, W. (1981), ACM SIGMOD

- Considerar como profundidad global $D = 3$
- El algoritmo inicia con dos buckets (0 y 1).
- Para ubicar un registro en su bucket correspondiente, tome los d-primeros dígitos del hash (d es el nivel del árbol).
- Cuando un bucket se desborda, cree dos nodos hijos incrementando el nivel de bits (por ejemplo, $01 \rightarrow 010, 011$)

1. Dibuje el árbol final.

Trie de directorio con $D = 3$: las hojas guardan el puntero al bucket



Las hojas quedaron en 000, 001, 010, 011, 10, 110, 111. La rama de la izquierda bajó hasta el tercer nivel porque las claves con hash 001 y 011 se apiñaron; la rama 10, en cambio, se quedó en el segundo nivel y nunca necesitó hijos. Un árbol deliberadamente desbalanceado, y está bien que lo esté: se profundiza donde hay densidad de claves y no donde no la hay.

2. Diseñe el algoritmo de inserción.

insertar(k, registro):

$h \leftarrow \text{Binary}(k \bmod 2^D)$

nodo $\leftarrow$ raíz; $d \leftarrow 0$

mientras nodo no sea hoja: # bajar consumiendo bits

nodo $\leftarrow$ hijo[$h[d]$]; $d \leftarrow d + 1$

$b \leftarrow \text{leer_bucket}(\text{nodo.puntero})$

si b tiene espacio: escribir y return

si $d < D$: # todavía se puede profundizar

convertir la hoja en nodo interno

crear dos hojas nuevas: $h||0$ y $h||1$, con un bucket cada una

repartir los registros de b según su bit d

reintentar la inserción de k # puede volver a partirse

sino: # $d = D$, no hay más bits que mirar

si $b.\text{overflow} == \text{NULL}$ y $K > 0$:

enganchar un bucket de overflow con k

sino:

REHASHING con $D = D + 1$

Lo de reintentar en vez de insertar directo importa: cuando las cuatro claves comparten el bit d , la partición deja todo de un lado y el bucket sigue lleno. Reintentando, el algoritmo vuelve a evaluar y parte otra vez si aún puede. Es lo que pasó con la entrada 00 de P2.

3. Diseñe el algoritmo de búsqueda.

buscar(k):

```
h <- Binary(k mod  $2^D$ )
nodo <- raíz; d <- 0
mientras nodo no sea hoja:
    nodo <- hijo[ h[d] ]; d <- d + 1
    si nodo == NULL: return NO_ESTA    # rama inexistente

b <- leer_bucket(nodo.puntero)    # única lectura a disco
si k está en b: return registro
si b.overflow != NULL:
    return buscar_en(b.overflow, k)
return NO_ESTA
```

El descenso por el trie no cuesta I/O: el árbol vive en RAM, son punteros. Lo único que toca disco es leer el bucket de la hoja

4. (Opcional) Diseñe el algoritmo de eliminación.

eliminar(k):

```
localizar la hoja como en buscar(k)
si k no está: return FALSE
quitar k; compactar la cadena; liberar el overflow si quedó vacío

# colapso del árbol, de abajo hacia arriba
p <- padre de la hoja
mientras p tenga sus dos hijos como hojas
    y |hijo_izq| + |hijo_der| <= FB:
        volcar los dos buckets en uno
        p pasa a ser hoja apuntando a ese bucket
        liberar el bucket sobrante
    p <- padre de p
```

El colapso es la operación inversa de la partición y solo se puede aplicar cuando los dos hijos son hojas: si uno es interno, sus descendientes viven más abajo y fusionar significaría rehacer todo un subárbol.

P4. Linear Hashing

Investigue cómo aplicar la técnica de Linear Hashing a nivel de buckets:

Linear Hashing: a new tool for file and table addressing. Litwin, W. (1980)

Litwin propuso esto en 1980 con una idea que, la primera vez que uno la lee, suena a trampa: crecer el archivo sin directorio y sin que el bucket que se parte tenga nada que ver con el bucket que se desbordó. Suena mal. Funciona sorprendentemente bien.

Los ingredientes son tres: el nivel L , que dice cuántas veces se duplicó el archivo; el puntero p , que marca cuál es el próximo bucket en la fila para partirse; y dos funciones hash encadenadas, $h_L(k) = k \bmod (N \cdot 2^L)$ y su versión con el doble de rango. Con $N = 2$ buckets iniciales y $FB = 3$, el direccionamiento queda así:

`dirección(k):`

```
i <- k mod (N * 2^L)
si i < p:           # ese bucket ya se partió en esta ronda
    i <- k mod (N * 2^(L+1))
return i
```

La política que usé para disparar el split es la clásica: cada vez que hay un desbordamiento se parte el bucket p , sea cual sea. También se puede disparar por factor de carga, y en un sistema real eso da mejor control de la ocupación, pero para 17 claves la regla del overflow es la que deja la traza más legible.

1. Elabore paso a paso el Linear Hashing usando los datos de la pregunta P2.
2. Diseñe el algoritmo de inserción.

`insertar(k, registro):`

```
i <- dirección(k)
b <- leer_bucket(i)
si b tiene espacio:
    escribir registro en b
sino:
    enganchar/usar bucket de overflow de la cadena de i

# el split va aparte, y siempre sobre p
base <- N * 2^L
nuevo <- p + base
repartir los registros de la cadena de p entre p y nuevo
    usando k mod (2 * base)
p <- p + 1
si p == base:           # se completó la ronda
    p <- 0; L <- L + 1
```

3. Diseñe el algoritmo de búsqueda.

buscar(k):

$i \leftarrow k \bmod (N * 2^L)$

 si $i < p$: $i \leftarrow k \bmod (N * 2^{(L+1)})$

$b \leftarrow \text{leer_bucket}(i)$

 mientras $b \neq \text{NULL}$:

 si k está en b : return registro

$b \leftarrow \text{leer_bucket}(b.\text{overflow})$

 return NO_ESTA

Dos operaciones aritméticas y una lectura. Sin índice que consultar, sin nivel que bajar, sin nada en RAM más que dos enteros: L y p .